

Engineering Reboot 2026

Day 21 Field Manual

Week 05 - Professional Python Foundations

Exception Handling Fundamentals

Publication Edition v3.2 - Rich Word-Formatted Manual

Program	Engineering Reboot 2026
Week	Week 05
Day	Day 21
Primary Project	Safe Calculator
Repository Target	01-Week-05/Exercises/Day-21 and 01-Week-05/Projects/Day-21
Publication Note This rich edition preserves the corrected v3.1 content while improving Word-native formatting so that PDF export retains the intended visual structure: cover page, tables, callouts, code panels, console output blocks, validation matrices, and appendix formatting.	

Revision History

Version	Date	Status	Notes
v3.2	June 28, 2026	Rich Publication Edition	Enhanced Word formatting designed for reliable PDF rendering while preserving Day 21 assignment content.

Table of Contents

1. Day 21 Mission
 2. Learning Objectives
 3. Repository and File Naming Standard
 4. Engineering Theory
 5. Exception Handling Syntax Reference
 6. Required Exercises
 7. Mini Project - Safe Calculator
 8. Evidence Requirements
 9. Validation Matrix
 10. Stretch Goals
 11. Notes and Journal Guidance
 12. Preview of Day 22
- Appendix A - Full Program Listing Index
- Appendix B - Common Errors and Troubleshooting

1. Day 21 Mission

Day 21 begins Week 05: Professional Python Foundations. The week shifts from simply making scripts run to designing software that behaves predictably under imperfect conditions. The technical focus for Day 21 is exception handling: anticipating operations that may fail, recovering from expected failures, and preserving a clean user experience.

Engineering Principle

A professional program does not assume perfect input or perfect runtime conditions. It defines recovery paths for expected failures and fails loudly only when a defect should be investigated.

2. Learning Objectives

- Explain what a Python exception is and when it occurs.
- Distinguish syntax errors from runtime exceptions.
- Use try, except, else, and finally in appropriate contexts.
- Catch specific built-in exceptions such as ValueError and ZeroDivisionError.
- Raise exceptions intentionally when business rules are violated.
- Build interactive programs that recover from invalid input.
- Produce evidence showing both successful and failing execution paths.

3. Repository and File Naming Standard

All Day 21 work must use explicit filenames. Do not create generic files such as main.py or exercise.py unless a project structure specifically calls for them. Each exercise file should be stored under the Day 21 exercise folder.

```
C:\Projects\Engineering-Reboot-2026\01-Week-05\Exercises\Day-21\  
value_error_demo.py  
zero_division_demo.py  
multiple_exceptions_demo.py  
try_else_demo.py  
finally_demo.py  
raise_exception_demo.py  
nested_exception_demo.py  
  
C:\Projects\Engineering-Reboot-2026\01-Week-05\Projects\Day-21\safe_calculator\  
main.py  
calculator.py  
input_helpers.py  
README.md
```

Filename Rule

The filename is part of the assignment. It creates traceability between the manual, the code, the screenshot

evidence, and the Git commit.

4. Engineering Theory

4.1 Syntax Errors vs Runtime Exceptions

A syntax error prevents Python from understanding the program. A runtime exception occurs while the program is running. Day 21 focuses on runtime exceptions, because these are the failures professional software must often anticipate and recover from.

4.2 Exception Lifecycle

- An operation fails, such as converting abc to a number.
- Python creates an exception object, such as ValueError.
- Python searches for a matching except block.
- If found, control transfers to the handler.
- If not found, the exception propagates up the call stack.
- If no handler exists, the program terminates and prints a traceback.

4.3 Narrow Exception Handling

Catching specific exceptions is preferred. A broad handler can hide real defects. For example, a ValueError caused by invalid user input is recoverable; a NameError caused by a misspelled variable is a programming defect and should not be silently hidden.

5. Exception Handling Syntax Reference

Best Practice

Keep the try block small. Protect only the operation that may fail. This makes the handler easier to reason about and prevents unrelated errors from being misclassified.

```
try:
    # Code that may fail
    result = risky_operation()
except ValueError:
    # Recovery path for one expected failure type
    print("Invalid value.")
except ZeroDivisionError:
    # Recovery path for another expected failure type
    print("Cannot divide by zero.")
else:
    # Runs only when no exception occurs
    print("Operation succeeded.")
finally:
    # Runs whether the operation succeeds or fails
    print("Operation complete.")
```

6. Required Exercises

Complete all seven exercises before starting the mini project. Each exercise includes a required filename, objective, implementation notes, starter solution, expected output, and evidence requirement.

Exercise 1 - Handling ValueError

Required filename	value_error_demo.py
Objective	Convert user input to a number and recover when the input is not numeric.
Screenshot evidence	value_error_demo-T01-Execution.png

Implementation

```
while True:
    user_input = input("Enter a number: ")
    try:
        number = float(user_input)
        print(f"Accepted value: {number}")
        break
    except ValueError:
        print("Invalid numeric input. Please try again.")
```

Expected Output

```
Enter a number: abc
Invalid numeric input. Please try again.
Enter a number: 12.5
Accepted value: 12.5
```

Engineering Notes

Verify both success and failure paths. The purpose of this exercise is not merely to avoid a crash; it is to provide predictable behavior and clear user feedback.

Exercise 2 - Handling ZeroDivisionError

Required filename	zero_division_demo.py
Objective	Perform division safely and recover when the denominator is zero.
Screenshot evidence	zero_division_demo-T01-Execution.png

Implementation

```
try:
    numerator = float(input("Enter numerator: "))
    denominator = float(input("Enter denominator: "))
    result = numerator / denominator
    print(f"Result: {result}")
except ValueError:
    print("Both values must be numeric.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Expected Output

```
Enter numerator: 10
Enter denominator: 0
Cannot divide by zero.
```

Engineering Notes

Verify both success and failure paths. The purpose of this exercise is not merely to avoid a crash; it is to provide predictable behavior and clear user feedback.

Exercise 3 - Multiple Exception Blocks

Required filename	multiple_exceptions_demo.py
Objective	Handle ValueError and ZeroDivisionError using separate exception handlers.
Screenshot evidence	multiple_exceptions_demo-T01-Execution.png

Implementation

```
try:
    first = float(input("First number: "))
    second = float(input("Second number: "))
    quotient = first / second
except ValueError:
    print("Invalid input: numbers are required.")
except ZeroDivisionError:
    print("Invalid operation: division by zero is not allowed.")
else:
    print(f"Quotient: {quotient}")
```

Expected Output

```
First number: 8
```

Second number: 2
Quotient: 4.0

Engineering Notes

Verify both success and failure paths. The purpose of this exercise is not merely to avoid a crash; it is to provide predictable behavior and clear user feedback.

Exercise 4 - Using else

Required filename	try_else_demo.py
Objective	Use else to separate success-path logic from exception handling logic.
Screenshot evidence	try_else_demo-T01-Execution.png

Implementation

```
try:
    age = int(input("Enter age: "))
except ValueError:
    print("Age must be a whole number.")
else:
    print(f"Age accepted: {age}")
```

Expected Output

Enter age: 53
Age accepted: 53

Engineering Notes

Verify both success and failure paths. The purpose of this exercise is not merely to avoid a crash; it is to provide predictable behavior and clear user feedback.

Exercise 5 - Using finally

Required filename	finally_demo.py
Objective	Demonstrate logic that runs regardless of success or failure.
Screenshot evidence	finally_demo-T01-Execution.png

Implementation

```
try:
    filename = input("Enter filename to open: ")
    file = open(filename, "r")
    print(file.read())
except FileNotFoundError:
    print("File was not found.")
finally:
    print("File operation attempt completed.")
```

Expected Output

```
Enter filename to open: missing.txt
File was not found.
File operation attempt completed.
```

Engineering Notes

Verify both success and failure paths. The purpose of this exercise is not merely to avoid a crash; it is to provide predictable behavior and clear user feedback.

Exercise 6 - Raising Exceptions

Required filename	raise_exception_demo.py
Objective	Raise ValueError intentionally when a business rule is violated.
Screenshot evidence	raise_exception_demo-T01-Execution.png

Implementation

```
def validate_age(age: int) -> None:
    if age < 0:
        raise ValueError("Age cannot be negative.")

try:
    age = int(input("Enter age: "))
    validate_age(age)
except ValueError as error:
    print(f"Invalid age: {error}")
else:
    print(f"Age accepted: {age}")
```

Expected Output

```
Enter age: -5
```

Invalid age: Age cannot be negative.

Engineering Notes

Verify both success and failure paths. The purpose of this exercise is not merely to avoid a crash; it is to provide predictable behavior and clear user feedback.

Exercise 7 - Nested Exception Handling

Required filename	nested_exception_demo.py
Objective	Combine validation loops and exception handling to build a resilient input workflow.
Screenshot evidence	nested_exception_demo-T01-Execution.png

Implementation

```
def read_number(prompt: str) -> float:
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Invalid number. Please try again.")

first = read_number("First number: ")
second = read_number("Second number: ")

try:
    result = first / second
except ZeroDivisionError:
    print("Cannot divide by zero.")
else:
    print(f"Result: {result}")
```

Expected Output

```
First number: x
Invalid number. Please try again.
First number: 9
Second number: 3
Result: 3.0
```

Engineering Notes

Verify both success and failure paths. The purpose of this exercise is not merely to avoid a crash; it is to

provide predictable behavior and clear user feedback.

7. Mini Project - Safe Calculator

The Day 21 mini project integrates all exception-handling concepts into a professional menu-driven console application. The project must use multiple files and must remain operational after recoverable errors.

7.1 Required Project Folder

```
C:\Projects\Engineering-Reboot-2026\01-Week-05\Projects\Day-21\safe_calculator\
main.py
calculator.py
input_helpers.py
README.md
```

7.2 Requirements

- Display a menu with Add, Subtract, Multiply, Divide, and Exit.
- Accept numeric input using a reusable helper function.
- Reject invalid menu selections without terminating.
- Handle invalid numeric input using ValueError.
- Handle division by zero using ZeroDivisionError or proactive validation.
- Continue running until the user selects Exit.
- Use clear function names and small functions.
- Capture screenshots for success and failure paths.

7.3 calculator.py

```
def add(a: float, b: float) -> float:
    return a + b

def subtract(a: float, b: float) -> float:
    return a - b

def multiply(a: float, b: float) -> float:
    return a * b

def divide(a: float, b: float) -> float:
    if b == 0:
        raise ZeroDivisionError("Division by zero is not allowed.")
    return a / b
```

7.4 input_helpers.py

```
def read_float(prompt: str) -> float:
```

```

while True:
    try:
        return float(input(prompt))
    except ValueError:
        print("Invalid numeric input. Please enter a valid number.")

def read_menu_choice() -> str:
    return input("Choose an option: ").strip()

```

7.5 main.py

```

from calculator import add, subtract, multiply, divide
from input_helpers import read_float, read_menu_choice

def display_menu() -> None:
    print("
Safe Calculator")
    print("1. Add")
    print("2. Subtract")
    print("3. Multiply")
    print("4. Divide")
    print("5. Exit")

def run_calculation(choice: str) -> None:
    first = read_float("Enter first number: ")
    second = read_float("Enter second number: ")

    try:
        if choice == "1":
            result = add(first, second)
        elif choice == "2":
            result = subtract(first, second)
        elif choice == "3":
            result = multiply(first, second)
        elif choice == "4":
            result = divide(first, second)
        else:
            print("Invalid menu selection.")
            return
    except ZeroDivisionError as error:
        print(f"Calculation error: {error}")
    else:
        print(f"Result: {result}")

def main() -> None:
    while True:
        display_menu()
        choice = read_menu_choice()
        if choice == "5":
            print("Exiting Safe Calculator.")
            break

```

```
run_calculation(choice)

if __name__ == "__main__":
    main()
```

7.6 Expected Project Output

```
Safe Calculator
1. Add
2. Subtract
3. Multiply
4. Divide
5. Exit
Choose an option: 4
Enter first number: 10
Enter second number: 0
Calculation error: Division by zero is not allowed.
```

```
Safe Calculator
1. Add
2. Subtract
3. Multiply
4. Divide
5. Exit
Choose an option: 1
Enter first number: 7
Enter second number: 8
Result: 15.0
```

7.7 Required Project Evidence Filenames

- Safe-Calculator-Code-main.png
- Safe-Calculator-Code-calculator.png
- Safe-Calculator-Code-input_helpers.png
- Safe-Calculator-T01-Addition-Success.png
- Safe-Calculator-T02-Invalid-Number-Recovery.png
- Safe-Calculator-T03-Divide-By-Zero-Recovery.png
- Safe-Calculator-T04-Invalid-Menu-Recovery.png
- Safe-Calculator-Git-Operations.png

8. Evidence Requirements

Evidence Item	Required Filename	Purpose
Exercise code screenshots	<program-name>-Code.png	Show completed source code.
Execution screenshots	<program-name>-T01-Execution.png	Show expected runtime behavior.

Failure-path screenshots	<program-name>-T02-Failure-Recovery.png	Show invalid input or exception recovery.
Git evidence	Git-Operations.png	Show status, add, commit, push, and verification.
Notes	Notes.md	Document concepts and commands.
Journal	Journal.md	Reflect on learning and issues encountered.

9. Validation Matrix

Area	Validation Question	Expected Result	Status
Exercises	Are all seven files created with required filenames?	Yes	☐
Exception Handling	Are ValueError and ZeroDivisionError demonstrated?	Yes	☐
Project	Does Safe Calculator continue after recoverable errors?	Yes	☐
Output	Do screenshots match expected outputs?	Yes	☐
Git	Was work committed and pushed?	Yes	☐
Documentation	Were Notes and Journal updated?	Yes	☐

10. Stretch Goals

- ☐ Create a custom exception class named CalculatorError.
- ☐ Log recoverable errors to calculator_errors.log.
- ☐ Add retry limits for numeric input.
- ☐ Handle KeyboardInterrupt gracefully.
- ☐ Add exponentiation and modulo operations.
- ☐ Write a short README explaining how to run the project.

11. Notes and Journal Guidance

Notes.md prompts

- Define exception, handler, traceback, propagation, finally, and raise.
- Summarize when to use try/except versus validation.
- Record the difference between ValueError and TypeError.

Journal.md prompts

- Which exception was easiest to understand?
- Which failure path was hardest to test?
- How did the Safe Calculator improve compared with earlier calculator versions?

12. Preview of Day 22

Day 22 extends Day 21 by shifting from recovery to prevention. You will design validation rules that reject bad input before it reaches business logic. Together, exception handling and validation form the foundation of defensive programming.

Appendix A - Full Program Listing Index

- `value_error_demo.py` - Convert user input to a number and recover when the input is not numeric.
- `zero_division_demo.py` - Perform division safely and recover when the denominator is zero.
- `multiple_exceptions_demo.py` - Handle `ValueError` and `ZeroDivisionError` using separate exception handlers.
- `try_else_demo.py` - Use `else` to separate success-path logic from exception handling logic.
- `finally_demo.py` - Demonstrate logic that runs regardless of success or failure.
- `raise_exception_demo.py` - Raise `ValueError` intentionally when a business rule is violated.
- `nested_exception_demo.py` - Combine validation loops and exception handling to build a resilient input workflow.

Appendix B - Common Errors and Troubleshooting

Problem	Likely Cause	Correction
Program still crashes on abc	Conversion is outside try block	Move <code>float()</code> or <code>int()</code> inside try.
Zero division not caught	Wrong exception type caught	Catch <code>ZeroDivisionError</code> or validate denominator.
Handler never runs	Exception occurs before try block	Move risky operation into try block.
All errors hidden	Bare except used	Catch specific exception types.
Program exits too early	No retry loop	Wrap input in <code>while True</code> and <code>break</code> only after success.